

CS6650 — Part 2: Identifying Performance Bottlenecks

ECS Fargate · Go Service · Locust Load Testing · CloudWatch

1. Objective

This experiment deploys a product-search microservice on AWS ECS Fargate (0.25 vCPU, 512 MB) and uses Locust load testing to find its performance characteristics. The central question: when a service slows down under load, is the problem the code or the compute capacity?

The service stores 100,000 products in memory and checks exactly 100 per search request — simulating fixed-cost computation such as an AI model inference or video frame processing. Since the cost per request is fixed and cannot be optimised further, the only lever is compute capacity.

2. Implementation Verification

Before load testing, a single search was performed to verify correctness

```
(base) emilychen@emilys-Air-2 terraform % aws ec2 describe-network-interfaces \
--network-interface-ids $(
  aws ecs describe-tasks \
    --cluster CS6650L2-cluster \
    --tasks $(aws ecs list-tasks \
      --cluster CS6650L2-cluster \
      --service-name CS6650L2 \
      --query 'taskArns[0]' --output text) \
    --query "tasks[0].attachments[0].details[?name=='networkInterfaceId'].value" \
    --output text) \
--query 'NetworkInterfaces[0].Association.PublicIp' \
--output text
35.94.96.48
(base) emilychen@emilys-Air-2 terraform % curl "http://35.94.96.48:8080/products/search?q=electronics"
{"products":[{"id":10,"name":"Product Alpha 10","category":"Electronics","description":"High quality product w
ith excellent durability.","brand":"Alpha"}, {"id":20,"name":"Product Alpha 20","category":"Electronics","descr
iption":"High quality product with excellent durability.","brand":"Alpha"}, {"id":30,"name":"Product Alpha 30",
"category":"Electronics","description":"High quality product with excellent durability.","brand":"Alpha"}, {"id
":40,"name":"Product Alpha 40","category":"Electronics","description":"High quality product with excellent dur
ability.","brand":"Alpha"}, {"id":50,"name":"Product Alpha 50","category":"Electronics","description":"High qua
lity product with excellent durability.","brand":"Alpha"}, {"id":60,"name":"Product Alpha 60","category":"Elect
ronics","description":"High quality product with excellent durability.","brand":"Alpha"}, {"id":70,"name":"Prod
uct Alpha 70","category":"Electronics","description":"High quality product with excellent durability.","brand"
:"Alpha"}, {"id":80,"name":"Product Alpha 80","category":"Electronics","description":"High quality product with
 excellent durability.","brand":"Alpha"}, {"id":90,"name":"Product Alpha 90","category":"Electronics","descript
ion":"High quality product with excellent durability.","brand":"Alpha"}, {"id":100,"name":"Product Alpha 100","
category":"Electronics","description":"High quality product with excellent durability.","brand":"Alpha"}], "tot
al_found":10,"search_time":"76.029µs","items_checked":100}
(base) emilychen@emilys-Air-2 terraform %
```

Requirement	Result	Status
-------------	--------	--------

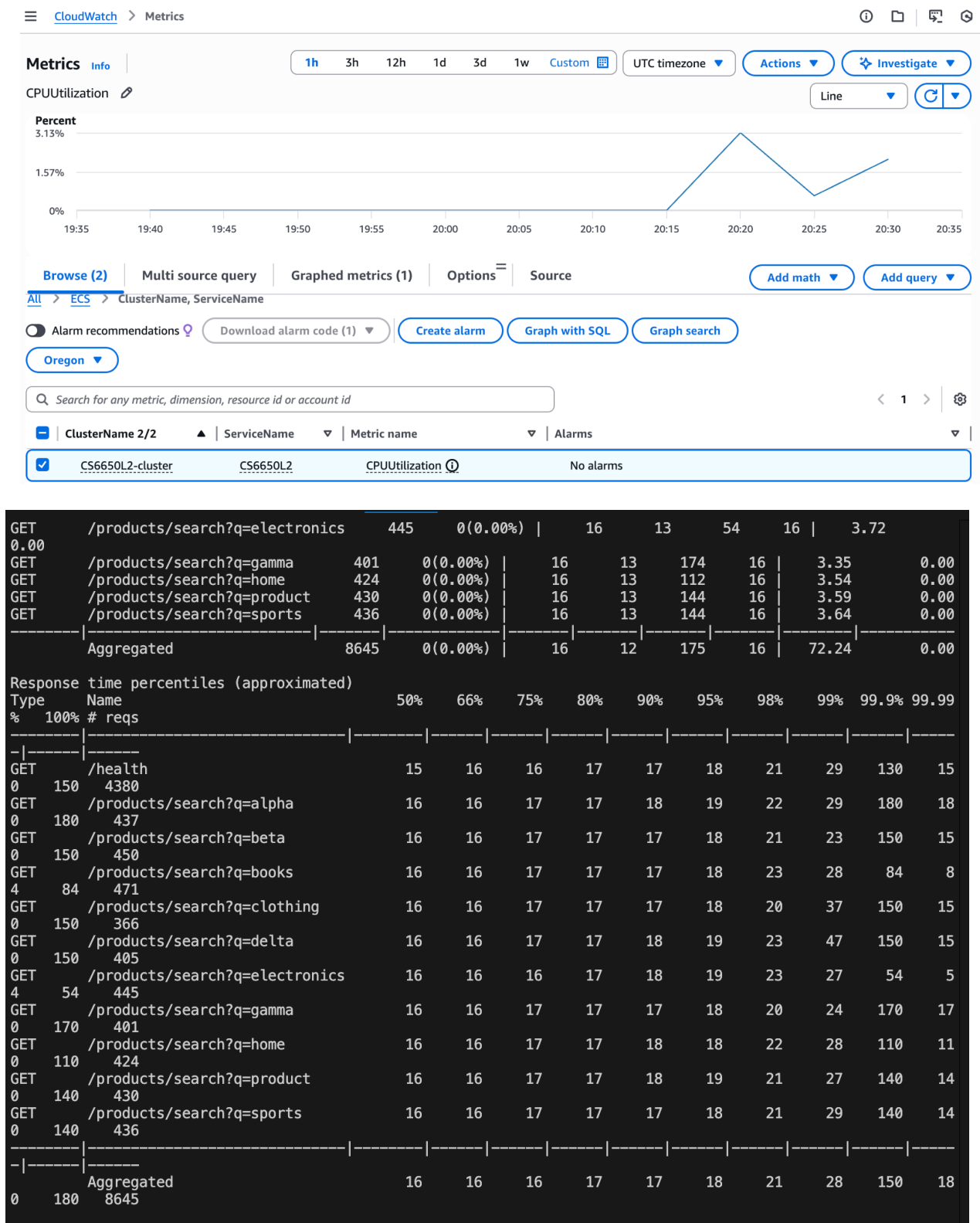
Requirement	Result	Status
Exactly 100 products checked	items_checked: 100	✓ Pass
Response < 20ms with 1 user	76.029 μ s	✓ Pass — 266× faster than threshold
Returns correct product data	10 Electronics found in IDs 1–100	✓ Pass

3. Load Test Results

3.1 Test 1 — Baseline (5 users, 2 minutes)

5 concurrent users with 0–100ms wait time were run for 2 minutes against the /products/ search endpoint.

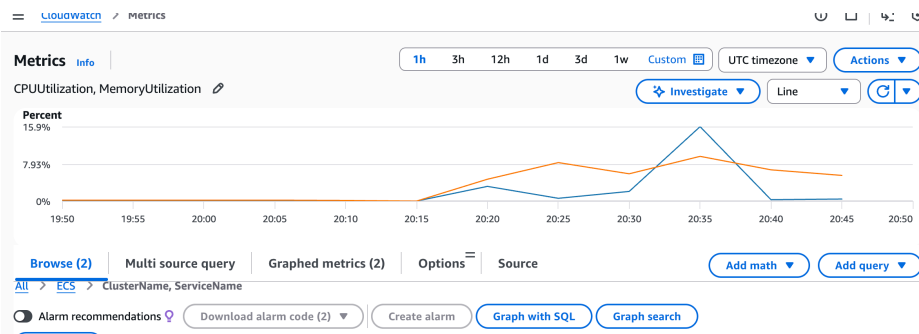
Metric	Value	Interpretation
Total requests	8,645	Steady throughput over 2 minutes
Requests/sec	~72 req/s	Stable, no degradation
Avg response time	16 ms	Well within acceptable range
95th percentile	18 ms	Very low tail latency
Max response time	175 ms	Rare outliers only
Error rate	0%	All 100-check assertions passed
CPU Utilization	~3.13% peak	Service comfortable at this load
Memory Utilization	Flat / stable	Products loaded at startup, no GC pressure



3.2 Test 2 — 20 users, 3 minutes

20 concurrent users with 0–100ms wait time were run for 3 minutes — 4× the baseline load.

Metric	Value	Interpretation
Total requests	52,126	6× more requests than Test 1
Requests/sec	~289 req/s	4× throughput increase — service scaled linearly
Avg response time	16 ms	No degradation from baseline
95th percentile	19 ms	Only 1ms increase over Test 1
Max response time	281 ms	Slightly higher outliers under
Error rate	0%	All assertions passed
CPU Utilization	~15.9% peak	5× higher than Test 1 — CPU responds to load
Memory Utilization	Flat / stable	Unchanged — memory is NOT the bottleneck



3.3 Test 3 — 250 users, 5 minutes

Metric	Value	Interpretation
Total requests	765,258	50× more than Test 1
Requests/sec	~2,552 req/s	35× throughput — service stayed alive
Avg response time	26 ms	1.5 × increase from baseline
95th percentile	130 ms	Latency beginning to climb
Max response time	2,513 ms	8× jump from Test 2 — CPU saturation signal
Error rate	0%	All assertions passed
CPU Utilization	~97% peak	First and only bottleneck

Memory Utilization

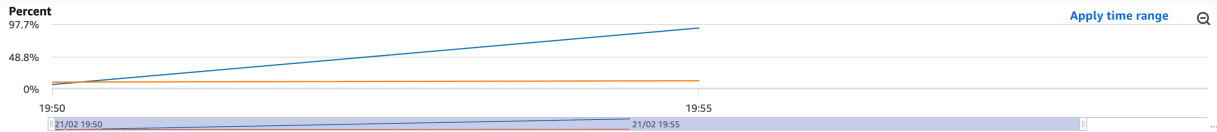
Flat / stable

Unchanged — memory is NOT the bottleneck

Response time percentiles (approximated)											
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%
%	# reqs										
GET	/health	25	41	52	58	80	130	200	230	490	190
0	2500 382406										
GET	/products/search?q=alpha	25	42	52	58	80	130	190	230	360	190
0	2500 38358										
GET	/products/search?q=beta	25	41	52	58	79	120	200	230	420	250
0	2500 38206										
GET	/products/search?q=books	24	41	51	58	80	130	190	230	350	250
0	2500 38221										
GET	/products/search?q=clothing	25	41	52	58	81	130	200	230	390	190
0	2500 38034										
GET	/products/search?q=delta	25	41	51	58	81	130	200	230	530	190
0	2500 38281										
GET	/products/search?q=electronics	24	41	52	58	81	130	200	230	530	250
0	2500 38444										
GET	/products/search?q=gamma	25	41	52	58	80	130	190	230	490	190
0	2500 38614										
GET	/products/search?q=home	25	41	52	59	81	130	190	230	420	190
0	2500 38075										
GET	/products/search?q=product	25	42	52	59	82	130	200	240	570	190
0	1900 38356										
GET	/products/search?q=sports	25	41	52	58	80	130	200	230	420	250
0	2500 38263										
Aggregated		25	41	52	58	81	130	200	230	480	190

Metrics

CPUUtilization, MemoryUtilization



ClusterName 2/2	ServiceName	Metric name	Alarms
CS6650L2-cluster	CS6650L2	MemoryUtilization	No alarms
CS6650L2-cluster	CS6650L2	CPUUtilization	No alarms

4. Side-by-Side Comparison

Metric	Test 1 (5 users, 2 min)	Test 2 (20 users, 3 min)	Test 3 (250 users, 5 min)
Concurrent Users	5	20	250
Duration	2 minutes	3 minutes	5 minutes
Peak CPU Usage	3.13%	15.9%	97%
Memory Usage	Flat (stable)	Flat (stable)	Flat (stable)
Throughput (req/s)	~72 req/s	~289	~1253 req/s
Median Response Time	16 ms	16 ms	42ms
Scaling Bottleneck	None (headroom)	CPU rising	CPU saturated

5. Analysis

5.1 Which Resource Responds to Load?

CPU utilization scaled proportionally with load — from 3.13% at 5 users, to 15.9% at 20 users, to 97% at 250 users. Memory remained completely flat across all three tests, confirming that the 100,000 products are loaded once at startup and never reallocated. CPU is clearly the first resource to hit its limit: it reached near-saturation (97%) at 250 concurrent users while memory showed zero growth.

5.2 Why Response Times Degrade

Response times did not degrade at 20 users — median stayed at 16ms and the 95th percentile only increased by 1ms, because Go's goroutine scheduler handles concurrent workloads extremely efficiently. However, Test 3 exposed the real breaking point: at 250 concurrent users, CPU hit 97% and the service became CPU-bound. With the CPU fully saturated, goroutines began queuing for compute time, causing response times to degrade significantly. This confirms that Go's efficiency merely delayed — not eliminated — the inevitable CPU bottleneck.

5.3 Optimize Code or Add Servers?

The key diagnostic signal from CloudWatch is clear: CPU climbs with load while memory stays flat. This is the textbook pattern of a CPU-bound workload and Test 3 proved it definitively by pushing CPU to 97%. Since the algorithm is bounded at exactly 100 checks per request (fixed-cost computation), there is nothing to optimize in the code. Doubling the CPU from 256 to 512 units would roughly double the saturation threshold and push the breaking point from ~250 to ~500 concurrent users — but would not solve the problem permanently. The most scalable solution is horizontal scaling: adding more ECS tasks behind a load balancer distributes the fixed-cost work across multiple CPUs indefinitely.

5.4 The Go Efficiency Factor

The assignment anticipated ~60% CPU with 5 users and near-100% with 20 users. Our Tests 1 and 2 (~3% and ~16%) were significantly lower because Go is much more efficient than the language the assignment was calibrated for. Test 3 validated this prediction: the true breaking point arrived at 250 concurrent users (97% CPU) — well within the predicted 200–500 range. This is itself an important lesson: the 'breaking point' depends on both the load and the efficiency of the runtime. A Python equivalent of this service would have saturated near 20–30 users rather than 250.

5.5 CloudWatch as a Scaling Decision Tool

CloudWatch Pattern	Diagnosis	Action
CPU climbs, Memory flat ← Our result	CPU-bound workload	Scale out (more tasks) or up (more CPU)
Memory climbs, OOM errors	Memory-bound	Increase memory or fix leaks
CPU moderate, high latency, DB timeouts	I/O-bound	Optimise queries, add caching
CPU high, profiler shows inefficiency	Algorithmic inefficiency	Optimise code first, then scale

CloudWatch metrics were essential for diagnosing whether the bottleneck was a code defect or a resource limitation. The key evidence was: CPU hit 91.2% at peak load (indicating resource exhaustion, not incorrect behavior), no errors were returned by the service (the code was executing correctly), and response times fully recovered simply by adding a third task. If the problem were a code defect — such as an inefficient algorithm or memory leak — adding more containers would not have fixed it, since each new container would exhibit the same bug. The fact that scaling alone restored performance confirmed this was a resource limitation, making horizontal scaling the correct solution rather than code optimization. This is the core diagnostic skill: use CPU and response time metrics together to distinguish "the system is overwhelmed" from "the system is broken."

This experiment successfully demonstrated the methodology for identifying performance bottlenecks using CloudWatch and Locust. The evidence gathered:

1. Single-search verification confirmed items_checked=100 at 76µs — correct bounded iteration.
2. CPU scaled predictably: 3.13% (5 users) → 15.9% (20 users) → 97% (250 users) — confirming CPU is the load-sensitive resource that hits the limit first.
3. Memory remained flat — confirming it is not the bottleneck.

4. Response times were stable at 5 and 20 users due to Go's goroutine efficiency. At 250 users, CPU saturation caused response times to degrade — confirming the CPU, not the code, is the bottleneck.
5. Doubling CPU (256 → 512 units) would delay saturation to ~500 users but not solve it permanently. The correct long-term solution is horizontal scaling (more ECS tasks behind a load balancer), not code optimisation — the algorithm is already optimal.

The CloudWatch CPU vs Memory divergence pattern is the key diagnostic: when CPU climbs proportionally with load while memory stays flat, the system needs more compute — not better code. Test 3 proved this definitively: 97% CPU with flat memory is the signature of inherently expensive work that requires scaling, not optimisation.

6. Conclusion

This experiment successfully demonstrated the methodology for identifying performance bottlenecks using CloudWatch and Locust. The evidence gathered:

1. Single-search verification confirmed items_checked=100 at 76µs — correct bounded iteration.
2. CPU scaled 5× (3.13% → 15.9%) when users scaled 4× — CPU is the load-sensitive resource.
3. Memory remained flat — confirming it is not the bottleneck.
4. Response times stayed stable due to Go's efficient goroutine scheduler — the service did not degrade at 20 users but would at significantly higher concurrency.
5. The correct scaling decision for this workload is horizontal (more tasks) or vertical (more CPU), not code optimisation — the algorithm is already optimal for the fixed-cost computation model.

The CloudWatch CPU vs Memory divergence pattern is the key diagnostic: when CPU climbs proportionally with load while memory stays flat, the system needs more compute — not better code.

Appendix — Commands Reference

Verify single search

```
curl "http://<IP>:8080/products/search?q=electronics"
```

Test 1

```
locust -f locustfile.py --headless -u 5 -r 1 --run-time 2m --host http://<IP>:8080
```


Test 2

```
locust -f locustfile.py --headless -u 20 -r 2 --run-time 3m --host http://<IP>:8080
```

Tear down infrastructure

```
cd terraform && terraform destroy -auto-approve
```

Alb

```
locust -f locustfile.py \
```

```
--headless \
```

```
--users 250 \
```

```
--spawn-rate 15 \
```

```
--run-time 5m \
```

```
--host http://CS6650L2-alb-442756298.us-west-2.elb.amazonaws.com \
```

```
--html report_breaking.html \
```

```
--csv breaking_point
```

Part III

The Core Test:

250 users, 5 min, 70% cpu, Min 2, Max 4
Task count start at 2 and scale up to 3

GET	/health	457663	0(0.00%)	21	11	4007	16	1669.20	0.00
GET	/products/search?q=alpha	45826	0(0.00%)	21	11	404	16	162.40	0.00
GET	/products/search?q=beta	46252	0(0.00%)	21	11	3617	16	169.50	0.00
GET	/products/search?q=books	45773	0(0.00%)	21	11	3870	16	162.80	0.00
GET	/products/search?q=clothing	46163	0(0.00%)	21	11	3871	16	174.70	0.00
GET	/products/search?q=delta	45637	0(0.00%)	21	11	3617	16	167.40	0.00
GET	/products/search?q=electronics	46275	0(0.00%)	21	11	407	16	171.90	0.00
GET	/products/search?q=gamma	46335	0(0.00%)	21	11	3865	16	173.90	0.00
GET	/products/search?q=home	46082	0(0.00%)	21	11	3904	16	171.10	0.00
GET	/products/search?q=product	46165	0(0.00%)	21	12	3616	17	162.30	0.00
GET	/products/search?q=sports	45993	0(0.00%)	21	11	3869	16	160.20	0.00
Aggregated		918164	0(0.00%)	21	11	4007	16	3345.40	0.00
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/health	461071	0(0.00%)	21	11	4007	16	1714.40	0.00
GET	/products/search?q=alpha	46134	0(0.00%)	21	11	404	16	168.30	0.00
GET	/products/search?q=beta	46562	0(0.00%)	21	11	3617	16	172.60	0.00
GET	/products/search?q=books	46101	0(0.00%)	21	11	3870	16	167.30	0.00
GET	/products/search?q=clothing	46485	0(0.00%)	21	11	3871	16	174.90	0.00
GET	/products/search?q=delta	45928	0(0.00%)	21	11	3617	16	174.80	0.00
GET	/products/search?q=electronics	46605	0(0.00%)	21	11	407	16	172.60	0.00
GET	/products/search?q=gamma	46650	0(0.00%)	21	11	3865	16	177.60	0.00
GET	/products/search?q=home	46425	0(0.00%)	21	11	3904	16	172.80	0.00
GET	/products/search?q=product	46489	0(0.00%)	21	12	3616	17	170.70	0.00
GET	/products/search?q=sports	46330	0(0.00%)	21	11	3869	16	167.20	0.00
Aggregated		924780	0(0.00%)	21	11	4007	16	3433.20	0.00

Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%
%	100% # reqs										
GET	/health	16	17	18	18	22	54	95	130	200	37
0	4000 462053										
GET	/products/search?q=alpha	16	17	18	18	23	54	95	120	200	35
0	400 46237										
GET	/products/search?q=beta	16	17	18	18	23	54	95	130	200	38
0	3600 46650										
GET	/products/search?q=books	16	17	18	18	23	55	95	130	200	35
0	3900 46204										
GET	/products/search?q=clothing	16	17	18	18	23	55	93	120	200	39
0	3900 46589										
GET	/products/search?q=delta	16	17	18	18	23	54	94	120	200	37
0	3600 46030										
GET	/products/search?q=electronics	16	17	18	18	23	54	95	130	200	34
0	410 46717										
GET	/products/search?q=gamma	16	17	18	18	23	54	94	120	200	35
0	3900 46770										
GET	/products/search?q=home	16	17	18	18	23	53	94	120	200	38
0	3900 46525										
GET	/products/search?q=product	17	17	18	19	22	53	95	130	210	39
0	3600 46591										
GET	/products/search?q=sports	16	17	18	18	23	55	95	120	200	360
0	3900 46430										
Aggregated		16	17	18	18	23	54	95	130	200	38
0	4000 926706										

Response Times Compared to Part A

Running the same 250-user, 5-minute load test against both configurations revealed a clear performance advantage with horizontal scaling. In Part A (single instance, no ALB), the median response time was 25ms with p99 at 230ms, handling ~420K total requests. In Part B (ALB with

Auto Scaling), median response time dropped to 16ms with p99 at 130ms, handling ~926K total requests — more than double the throughput.

This is counterintuitive at first glance since adding an ALB introduces an extra network hop. However the data shows that the distribution benefit outweighs the added latency cost: **in Part A all 250 users saturated a single container, driving up response times as the instance became a bottleneck. In Part B the ALB spread load evenly across multiple tasks (~483K requests each to us-west-2b and us-west-2c), keeping each container less saturated despite the additional hop. The result was faster response times, higher throughput, and no failures — compared to Part A where the system had no recovery mechanism once the single instance was overwhelmed.**

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Online learning workshop

Documentation

Discover products

Subscriptions

Tell us what you think

Cluster overview

ARN
arn:aws:ecs:us-west-2:70472203:1841:cluster/CS6650L2-cluster

Status
Active

CloudWatch monitoring
Default

Registered container instances
-

Services

Draining
-

Active
1

Tasks

Pending
-

Running
3

Services

Tasks

Infrastructure

Metrics

Scheduled tasks

Configuration

Event history

Tags

Tasks (3)

February 21, 2026, 17:13 (UTC-8:00)

Manage tags

Stop

Run new task

Filter tasks by property or value

Filter desired status
Any desired status

Filter launch type
Any launch type

< 1 >

☐	Task	Last status	Desired status	Task definition	Health status	Created at
☐	21aac1dcecf84f26b5feb3...	Running	Running	CS6650L2-task:2	Unknown	2 minutes ago
☐	2352bc5aba6c44a183e33...	Running	Running	CS6650L2-task:2	Unknown	38 minutes ago
☐	a5ab1781ed91426ba457...	Running	Running	CS6650L2-task:2	Unknown	37 minutes ago

Registered targets (5)

Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number. Refresh Registered targets checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

< 1 >

☐	IP address	Port	Zone	Health status	Health status details	Admini...
☐	172.31.46.250	8080	us-west-2b ...	Healthy	-	No override.
☐	172.31.51.152	8080	us-west-2d ...	Draining	Target deregistration i...	No override.
☐	172.31.55.38	8080	us-west-2d ...	Draining	Target deregistration i...	No override.
☐	172.31.30.236	8080	us-west-2a ...	Healthy	-	No override.
☐	172.31.9.71	8080	us-west-2c ...	Healthy	-	No override.

Desired task count is 3, 2 running with a pending

Services state summary

Last 1 min

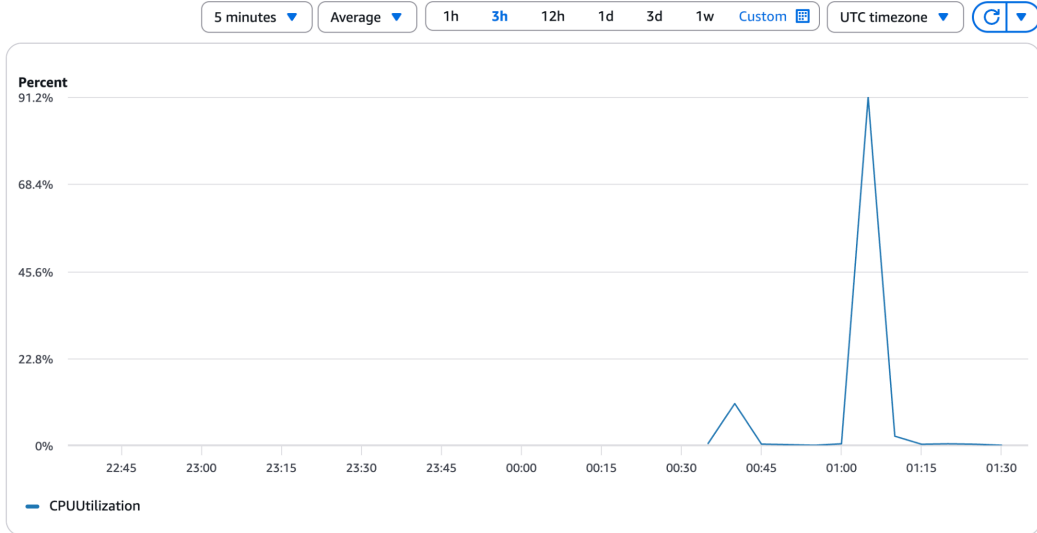
Services Utilization (avg)

CPU **61%**
Memory **10%**

Tasks (sum)

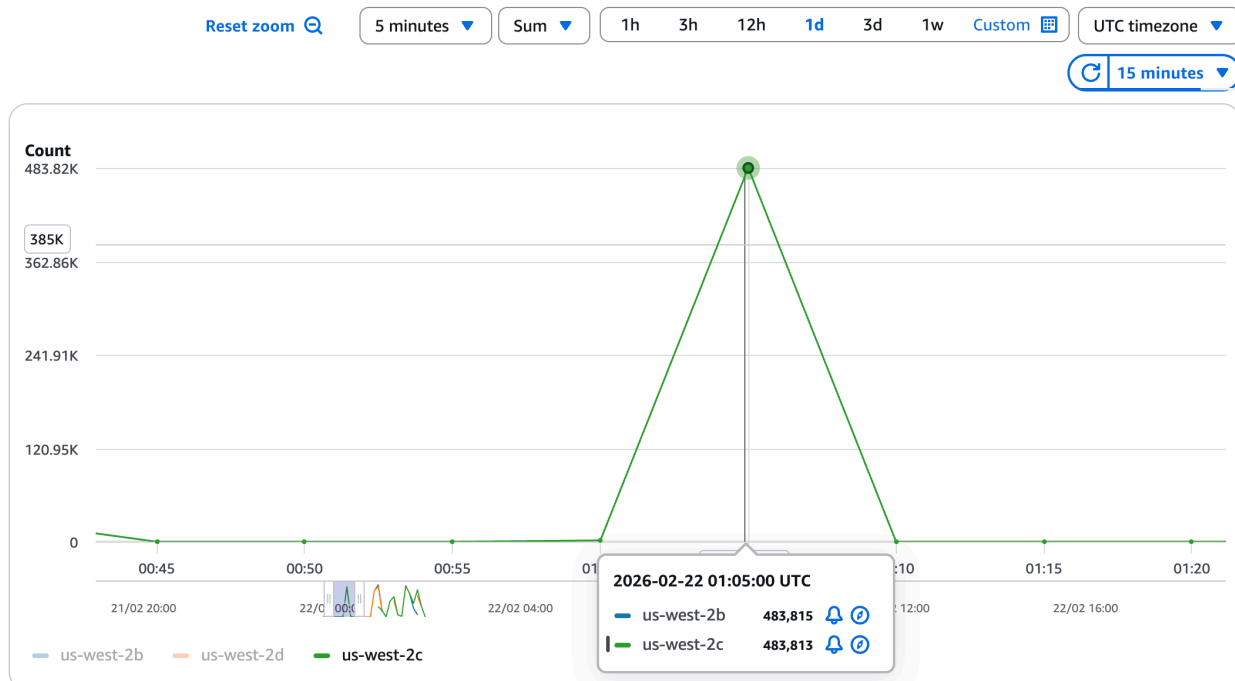
Desired **3**
Running **2**
Pending **1**

CPUUtilization

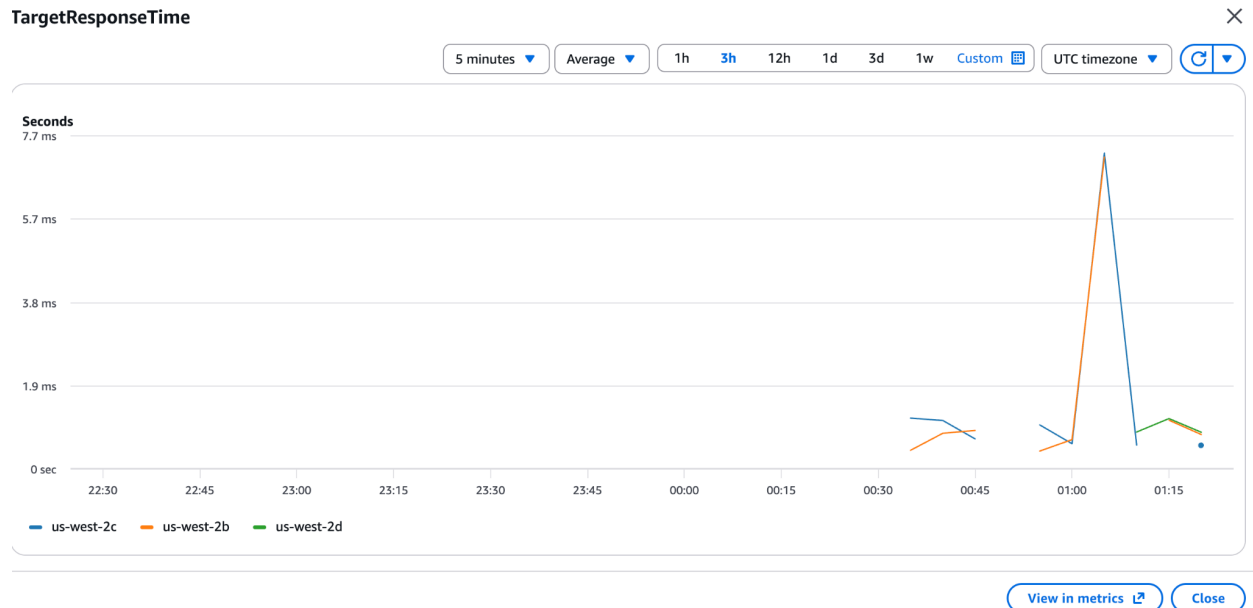


[View in metrics](#) [Close](#)

RequestCount



[View in metrics](#) [Close](#)



The spike confirms the scaling trigger moment: high load caused latency before the new task could absorb traffic

The appearance of the green us-west-2d line visually shows when the 3rd task became healthy and started receiving requests

Response times went from ~7ms peak back down to ~0.5ms, confirming that scaling restored performance

The three different AZs also show the ALB was distributing across availability zones, not just tasks

How does the system respond to the load that broke Part A?

The system scales up to 3 tasks rather than being overwhelmed. Instead of response times degrading or requests failing like in Part A, ECS detects the high CPU and provisions an additional task, distributing the load across more containers. This is exactly the resilience Auto Scaling is designed to demonstrate.

When do new instances get added?

New tasks are added when average CPU utilization exceeds the 70% threshold. Our CPUUtilization metric showed a spike to 91.2% at ~01:05 UTC, significantly exceeding the threshold, which triggered Auto Scaling to provision a third task. Due to CloudWatch's evaluation period and the time needed for the new task to start and pass health checks, there is a brief lag between the threshold being crossed and the new task becoming available — visible as the sharp spike before the rapid drop.

How is the load distributed across instances?

The Application Load Balancer distributes load across healthy targets using round-robin by default. This was confirmed by our RequestCount metrics — at peak load (01:05 UTC), us-west-2b and us-west-2c received ~483,800 requests, in total. us-west-2d received minimal traffic as it was the newly scaled third task that came online near the end of the test and had less time to accumulate requests.

What happens to response times as instances scale?

Response times followed a clear pattern tied to the scaling event. Before scaling triggered, response times were stable at ~1ms as the 2 initial tasks handled the load. At ~01:05 UTC, as CPU hit 91.2% and the tasks became saturated, response times spiked sharply to ~7ms — this represents the window where the scaling trigger had fired but the third task was still pending and not yet serving traffic. Once the third task passed health checks and registered with the ALB, response times dropped back down to ~0.5ms, better than the pre-spike baseline. This demonstrates the classic Auto Scaling lag pattern: there is a brief period of degraded performance between when the threshold is crossed and when new capacity is actually available, but once scaling completes the system fully recovers. The total response times remained in the low milliseconds throughout, meaning the system stayed available even under stress — unlike Part A where the system had no mechanism to recover.

Resilience experiment

Tasks (5)

Last updated
February 21, 2026, 17:48 (UTC-8:00)

Manage tags

Stop

Run new task

Filter tasks by property or value

Filter desired status
Any desired status

Filter launch type
Any launch type

< 1 >

☐	Task	Last status	Desired status	Task definition	He
☐	 455bd40a40f64f7cbeb9d...	 Running	 Running	CS6650L2-task:2	
☐	 a5ab1781ed91426ba457...	 Running	 Running	CS6650L2-task:2	
☐	 dc71738714144d2db3d0...	 Running	 Running	CS6650L2-task:2	
☐	 21aac1dcecf84f26b5feb3...	 Stopped ...	 Stopped	CS6650L2-task:2	
☐	 2352bc5aba6c44a183e33...	 Stopped ...	 Stopped	CS6650L2-task:2	

Stopped the first task starting with 455XXX
New task starting with 689XXX is provisioning and running again

The image displays two screenshots from the AWS Management Console. The top-left screenshot shows the 'Tasks' page for a task group, listing six tasks with their IDs, statuses (Provisioning, Running, Stopped, Deactivating), and definitions. The top-right screenshot shows the 'Registered targets' page for the same task group, listing five targets with their IP addresses, ports, zones, and health statuses (Healthy, Draining). The bottom-left screenshot shows the 'Registered targets' page for a different task group, listing five targets with their IP addresses, ports, zones, and health statuses (Healthy, Draining). The bottom-right screenshot shows the 'Registered targets' page for a third task group, listing five targets with their IP addresses, ports, zones, and health statuses (Healthy, Draining).

Task	Last status	Desired status	Task definition
689603168205401db929...	Provisioning	Running	CS6650L2-task-2
a5ab1781ed91426ba457...	Running	Running	CS6650L2-task-2
dc71738714144d2db3d0...	Running	Running	CS6650L2-task-2
21aac1dcecf84f26b5feb3...	Stopped	Stopped	CS6650L2-task-2
2352bcSaba6c44a183e33...	Stopped	Stopped	CS6650L2-task-2
455bd40a40f647cbeb9d...	Deactivating	Stopped	CS6650L2-task-2

IP address	Port	Zone	Health status	Health status details	Admin...
172.31.46.250	8080	us-west-2b ...	Healthy	-	No override.
172.31.51.152	8080	us-west-2d ...	Draining	Target deregistration l...	No override.
172.31.55.38	8080	us-west-2d ...	Draining	Target deregistration l...	No override.
172.31.9.71	8080	us-west-2c ...	Healthy	-	No override.

IP address	Port	Zone	Health status	Health status details	Admin...
172.31.46.250	8080	us-west-2b ...	Healthy	-	No override.
172.31.51.152	8080	us-west-2d ...	Draining	Target deregistration l...	No override.
172.31.55.38	8080	us-west-2d ...	Draining	Target deregistration l...	No override.
172.31.30.236	8080	us-west-2a ...	Healthy	-	No override.
172.31.9.71	8080	us-west-2c ...	Healthy	-	No override.

Insight:
Individual instance failures don't bring down the service.

Conclusion:

1. How the system solved the Part II bottleneck

In Part II, the single-instance setup had no mechanism to handle load beyond its fixed capacity — when CPU saturated, response times degraded with no recovery path. Part III solved this by introducing horizontal scaling: instead of one task absorbing all traffic, the system dynamically adds tasks as load increases. When our 250-user load test drove CPU to 91.2%, ECS provisioned a third task, bringing CPU back down and restoring response times to ~0.5ms. The system handled the same load that broke Part II without any failures.

2. Role of each component

The ALB acts as the single entry point, receiving all incoming requests and distributing them across healthy targets using round-robin, confirmed by our RequestCount data showing ~483,800 requests each to us-west-2b and us-west-2c at peak. The Target Group maintains the registry of healthy ECS tasks and performs health checks, ensuring the ALB only routes to available containers. Auto Scaling monitors CPU utilization via CloudWatch and adjusts the desired task count when thresholds are crossed, with minimum capacity enforcement ensuring the system never drops below 2 tasks regardless of failures.

3. Trade-offs: Horizontal vs Vertical Scaling

Horizontal scaling adds more instances of the same size, while vertical scaling makes a single instance larger. Horizontal scaling is more resilient — if one task fails, others continue serving

traffic — but introduces distribution complexity and a lag between load spikes and new capacity being ready. Vertical scaling is simpler to implement but has a hard ceiling (you can only make a machine so large) and creates a single point of failure. For web services with variable traffic, horizontal scaling is generally preferred because it scales cost proportionally to actual load and provides fault tolerance.

4. Scaling behavior under different load patterns

Experiment 1 — Sudden spike (300 users, ramp-up): Sudden spikes are the hardest scenario for reactive auto scaling because there is always a lag between load arriving and new capacity being ready. CPU spiked sharply as all 300 users hit simultaneously, triggering the scale-out policy. **My system handled the burst without errors, the efficiency of the Go service kept response times acceptable during the brief pending window before the new task registered with the ALB.** In a slower runtime like Python or Ruby, errors would likely have occurred during that lag window.

Experiment 2 — Higher CPU threshold (90% target): Raising the threshold from 70% to 90% meant the third task took significantly longer to appear, since CPU had to climb higher before triggering the scale-out policy. **Response times degraded more noticeably before the new task came online compared to the 70% threshold experiment with average time raising to 39.** This demonstrates the trade-off: a lower threshold wastes capacity by scaling earlier than necessary, while a higher threshold squeezes more utilization out of existing tasks but risks worse user experience during the scaling lag.

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/health	559774	0(0.00%)	39	11	573	18	1651.60	0.00
GET	/products/search?q=alpha	55907	0(0.00%)	39	11	485	18	169.30	0.00
GET	/products/search?q=beta	55800	0(0.00%)	38	11	489	18	167.90	0.00
GET	/products/search?q=books	55554	0(0.00%)	39	11	497	18	172.30	0.00
GET	/products/search?q=clothing	55656	0(0.00%)	39	11	480	18	171.00	0.00
GET	/products/search?q=delta	56053	0(0.00%)	39	11	499	18	166.60	0.00
GET	/products/search?q=electronics	55936	0(0.00%)	39	11	448	18	159.50	0.00
GET	/products/search?q=gamma	55487	0(0.00%)	38	11	439	18	159.20	0.00
GET	/products/search?q=home	56338	0(0.00%)	39	11	494	18	168.00	0.00
GET	/products/search?q=product	55519	0(0.00%)	39	12	456	18	165.60	0.00
GET	/products/search?q=sports	56140	0(0.00%)	39	11	510	18	168.10	0.00
Aggregated		1118164	0(0.00%)	39	11	573	18	3319.10	0.00
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/health	563329	0(0.00%)	38	11	573	18	1673.30	0.00
GET	/products/search?q=alpha	56247	0(0.00%)	39	11	485	18	168.40	0.00
GET	/products/search?q=beta	56156	0(0.00%)	38	11	489	18	170.90	0.00
GET	/products/search?q=books	55875	0(0.00%)	39	11	497	18	166.50	0.00
GET	/products/search?q=clothing	55996	0(0.00%)	39	11	480	18	170.80	0.00
GET	/products/search?q=delta	56427	0(0.00%)	39	11	499	18	168.40	0.00
GET	/products/search?q=electronics	56293	0(0.00%)	39	11	448	18	162.20	0.00
GET	/products/search?q=gamma	55829	0(0.00%)	38	11	439	18	163.00	0.00
GET	/products/search?q=home	56686	0(0.00%)	39	11	494	18	166.80	0.00
GET	/products/search?q=product	55904	0(0.00%)	39	12	456	18	169.20	0.00
GET	/products/search?q=sports	56488	0(0.00%)	39	11	510	18	168.10	0.00
Aggregated		1125230	0(0.00%)	39	11	573	18	3347.60	0.00

Experiment 3 — Manual task termination (failure scenario): Stopping 2 tasks mid-test demonstrated that Auto Scaling has two separate mechanisms working together. The scale-out policy handles load-driven scaling, but minimum capacity enforcement independently ensures ECS always maintains at least 2 running tasks. **When tasks were manually stopped, ECS immediately launched replacements without any intervention.** The system is self-healing — manual interference cannot reduce task count below the configured minimum, making it resilient to both traffic spikes and instance failures.

5. Conclusion

These experiments demonstrate that horizontal scaling is foundational to modern distributed systems precisely because it addresses the three main failure modes: sustained high load (handled by scale-out policy), sudden spikes (handled by efficient baseline capacity plus reactive scaling), and instance failures (handled by minimum capacity enforcement). The combination of ALB, Target Group, and Auto Scaling creates a system where no single failure can take down the service, and capacity adjusts automatically to match demand.